



G L O B A L R A I N

Artemis Financial Vulnerability Assessment Report

Table of Contents

ARTEMIS FINANCIAL VULNERABILITY ASSESSMENT REPORT	1
Document Revision History.....	3
Client.....	3
Developer.....	4
1. Interpreting Client Needs	4
2. Areas of Security.....	5
3. Manual Review	5
4. Static Testing.....	6
5. Mitigation Plan	10

Document Revision History

Version	Date	Author	Comments
1.0	[Date]	Jarrale Butts	

Client



Developer
Jarrale Butts

1. Interpreting Client Needs

- a) The value of secure communications to Artemis Financial would be rated as high. Artemis Financial, a consultancy specializing in sensitive financial data, places significant importance on secure communication. Upholding the secrecy and reliability of client information is essential for fostering trust and adhering to industry regulations.
- b) Artemis Financial doesn't explicitly state that it makes any international transactions. On the other hand, Global Rain, the company I work for (a client of Artemis Financial) creates software for entrepreneurs, businesses, and government agencies around the world. Artemis Financial might make international transactions, which could mean they might face more security issues when it comes to protecting data across borders and following different international privacy laws.
- c) There are possible governmental restrictions Artemis Financial may need to take into consideration. Considering government-imposed limitations concerning secure communication, data encryption, and data storage is crucial, particularly if Artemis Financial operates in multiple countries with diverse regulatory demands.
- d) There are always external threats that may be present now and in the immediate future. These threats include but are not limited to cyberattacks, data breaches, ransomware attacks, and denial-of-service attacks.
- e) It is important to keep modernization requirements in mind when developing software such as:
 - i. Open-source libraries offer an excellent means of enhancing applications with new features and capabilities. However, it's vital to thoroughly evaluate open-source libraries prior to integration, as they could potentially harbor security weaknesses.
 - ii. Web application technologies are in a state of continuous evolution, underscoring the importance for companies to ensure their applications remain current. This

entails adopting the most recent iterations of web servers, frameworks, and other essential software components.

2. Areas of Security

Using what I've learned in Step 1 and referring to the Vulnerability Assessment Process Flow Diagram I've concluded the areas of security include Input Validation, Cryptography, Client/Server, Code Error, and Code Quality. Specifically, Input Validation, Cryptography, and Client/Server are necessary for the code provided, though it is always good practice to pay attention to detail pertaining to Code Error and Code Quality.

3. Manual Review

I will refer to the seven security areas outlined in the Vulnerability Assessment Process Flow Diagram including Input Validation, APIs, Cryptography, Client/Server, Code Error, Code Quality, and Encapsulation for my manual review in the Project One Code Base. I separate my review by each class.

i. CRUD

- a. Input validation: The input parameters to the content and content2 fields are not validated.
- b. Cryptography: content and content2 fields are not encrypted.
- c. Client/server: The getContent() and getContent2() methods return the content and content2 fields without any authentication or authorization.

ii. CRUDController

- a. Input validation: The CRUDController.read() method does not validate the input parameter (name).
- b. Client/server: The CRUDController.read() method does not perform any authentication or authorization checks.

iii. Customer

- a. Input Validation: The deposit() method doesn't validate the input parameter (a) to guarantee it's a non-negative value.

- b. Cryptography: Encryption should be implemented for the customer's account number and account balance.
 - c. Client/Server: To enhance security, the showInfo() method should only reveal the customer's account number when the user's authentication and authorization are confirmed.
 - iv. DocData
 - a. Input validation: The read_document() method does not validate the input parameters (key and value).
 - b. Cryptography: The connection credentials to the database are stored in plain text in the code. This means that an attacker who gains access to the code could easily steal this information and access the database.
 - c. Client/server: The read_document() method does not perform any authentication or authorization checks.
 - v. Greeting
 - a. Input validation: The Greeting constructor does not validate the input parameters (id and content).
 - b. Cryptography: The Greeting class does not encrypt the content field.
 - vi. GreetingController
 - a. Input validation: The GreetingController.greeting() method does not validate the input parameter (name).
 - b. Client/server: The GreetingController.greeting() method does not perform any authentication or authorization checks.
 - vii. myDateTime
 - a. None I could identify.
 - viii. RestServiceApplication
 - a. None I could identify.

4. Static Testing

Code	Description	Attribution
bcprov-jdk15on-1.46.jar	The Bouncy Castle Crypto package is a Java implementation of cryptographic algorithms. This jar contains JCE provider and lightweight API for the Bouncy Castle Cryptography APIs for JDK 1.5 to JDK 1.7.	In Bouncy Castle JCE Provider version 1.55 and earlier the DSA does not fully validate ASN.1 encoding of signature on verification. It is possible to inject extra elements in the sequence making up the signature and still have it validate, which in some cases may allow the introduction of 'invisible' data into a signed structure.
classmate-1.5.1.jar	Library for introspecting types with full generic information including resolving of field and method types.	
hibernate-validator-6.0.18.Final.jar	Hibernate's Bean Validation (JSR-380) reference implementation.	A flaw was found in Hibernate Validator version 6.1.2.Final. A bug in the message interpolation processor enables invalid EL expressions to be evaluated as if they were valid. This flaw allows attackers to bypass input sanitation (escaping, stripping) controls that developers may have put in place when handling user-controlled data in error messages.
jackson-core-2.10.2.jar	Core Jackson processing abstractions (aka Streaming API), implementation for JSON	
jackson-databind-2.10.2.jar	General data-binding functionality for Jackson: works on core streaming API	A flaw was found in FasterXML Jackson Databind, where it did not have entity expansion secured properly. This flaw allows vulnerability to XML external entity (XXE) attacks. The highest threat from this vulnerability is data integrity.
jakarta.annotation-api-1.3.5.jar	Jakarta Annotations API	
jakarta.validation-api-2.0.2.jar	Jakarta Bean Validation API	
jboss-logging-3.4.1.Final.jar	The JBoss Logging Framework	
jul-to-slf4j-1.7.30.jar	JUL to SLF4J bridge	
log4j-api-2.12.1.jar	The Apache Log4j API	Improper validation of certificate with host mismatch in Apache Log4j SMTP appender. This could allow an SMTPS connection to be intercepted by a man-in-the-middle attack which could leak any log messages sent through that appender. Fixed in Apache Log4j 2.12.3 and 2.13.1

log4j-to-slf4j-2.12.1.jar	The Apache Log4j binding between Log4j 2 API and SLF4J.	
logback-core-1.2.3.jar	logback-core module	In logback version 1.2.7 and prior versions, an attacker with the required privileges to edit configurations files could craft a malicious configuration allowing to execute arbitrary code loaded from LDAP servers. CVE-2015-7502 Deserialization of Untrusted Data
slf4j-api-1.7.30.jar	The slf4j API	
snakeyaml-1.25.jar	YAML 1.1 parser and emitter for Java	SnakeYaml's Constructor() class does not restrict types which can be instantiated during deserialization. Deserializing yaml content provided by an attacker can lead to remote code execution. We recommend using SnakeYaml's SafeConstructor when parsing untrusted content to restrict deserialization. We recommend upgrading to version 1.26.0 or later.
spring-boot-2.2.4.RELEASE.jar	Spring Boot	In Spring Boot versions 3.0.0 - 3.0.5, 2.7.0 - 2.7.10, and older unsupported versions, an application that is deployed to Cloud Foundry could be susceptible to a security bypass. Users of affected versions should apply the following mitigation: 3.0.x users should upgrade to 3.0.6+. 2.7.x users should upgrade to 2.7.11+. Users of older, unsupported versions should upgrade to 2.0.9 or 2.7.11.
spring-boot-starter-web-2.2.4.RELEASE.jar	Starter for building web, including RESTful, applications using Spring MVC. Uses Tomcat as the default embedded container	In Spring Boot versions 3.0.0 - 3.0.5, 2.7.0 - 2.7.10, and older unsupported versions, an application that is deployed to Cloud Foundry could be susceptible to a security bypass. Users of affected versions should apply the following mitigation: 3.0.x users should upgrade to 3.0.6+. 2.7.x users should upgrade to 2.7.11+. Users of older, unsupported versions should upgrade to 2.0.9 or 2.7.11.
spring-core-5.2.3.RELEASE.jar	Spring Core	A Spring MVC or Spring WebFlux application running on JDK 9+ may be vulnerable to remote code execution (RCE) via data binding. The specific exploit requires the application to run on Tomcat as a WAR deployment. If the application is deployed as a Spring Boot executable jar, i.e. the default, it is not vulnerable to the exploit. However, the nature of the vulnerability is more general, and there may be other ways to exploit it.
spring-web-5.2.3.RELEASE.jar	Spring Web	Pivotal Spring Framework through 5.3.16 suffers from a potential remote code execution (RCE) issue if used for Java deserialization of untrusted data. Depending on how the library is implemented within a product, this issue may or not occur, and authentication may be required. NOTE: the vendor's position is that untrusted data is not an intended use case. The product's behavior will not be changed because some users rely on deserialization of trusted data.

spring-webmvc-5.2.3.RELEASE.jar	Spring Web MVC	A Spring MVC or Spring WebFlux application running on JDK 9+ may be vulnerable to remote code execution (RCE) via data binding. The specific exploit requires the application to run on Tomcat as a WAR deployment. If the application is deployed as a Spring Boot executable jar, i.e. the default, it is not vulnerable to the exploit. However, the nature of the vulnerability is more general, and there may be other ways to exploit it.
tomcat-embed-core-9.0.30.jar	Core Tomcat implementation	When using the Apache JServ Protocol (AJP), care must be taken when trusting incoming connections to Apache Tomcat. Tomcat treats AJP connections as having higher trust than, for example, a similar HTTP connection. If such connections are available to an attacker, they can be exploited in ways that may be surprising. In Apache Tomcat 9.0.0.M1 to 9.0.0.30, 8.5.0 to 8.5.50 and 7.0.0 to 7.0.99, Tomcat shipped with an AJP Connector enabled by default that listened on all configured IP addresses. It was expected (and recommended in the security guide) that this Connector would be disabled if not required. This vulnerability report identified a mechanism that allowed: - returning arbitrary files from anywhere in the web application - processing any file in the web application as a JSP Further, if the web application allowed file upload and stored those files within the web application (or the attacker was able to control the content of the web application by some other means) then this, along with the ability to process a file as a JSP, made remote code execution possible. It is important to note that mitigation is only required if an AJP port is accessible to untrusted users. Users wishing to take a defence-in-depth approach and block the vector that permits returning arbitrary files and execution as JSP may upgrade to Apache Tomcat 9.0.31, 8.5.51 or 7.0.100 or later. A number of changes were made to the default AJP Connector configuration in 9.0.31 to harden the default configuration. It is likely that users upgrading to
tomcat-embed-el-9.0.30.jar	Core Tomcat implementation	

tomcat-embed-websocket-9.0.30.jar	Core Tomcat implementation	When using the Apache JServ Protocol (AJP), care must be taken when trusting incoming connections to Apache Tomcat. Tomcat treats AJP connections as having higher trust than, for example, a similar HTTP connection. If such connections are available to an attacker, they can be exploited in ways that may be surprising. In Apache Tomcat 9.0.0.M1 to 9.0.0.30, 8.5.0 to 8.5.50 and 7.0.0 to 7.0.99, Tomcat shipped with an AJP Connector enabled by default that listened on all configured IP addresses. It was expected (and recommended in the security guide) that this Connector would be disabled if not required. This vulnerability report identified a mechanism that allowed: - returning arbitrary files from anywhere in the web application - processing any file in the web application as a JSP Further, if the web application allowed file upload and stored those files within the web application (or the attacker was able to control the content of the web application by some other means) then this, along with the ability to process a file as a JSP, made remote code execution possible. It is important to note that mitigation is only required if an AJP port is accessible to untrusted users. Users wishing to take a defence-in-depth approach and block the vector that permits returning arbitrary files and execution as JSP may upgrade to Apache Tomcat 9.0.31, 8.5.51 or 7.0.100 or later. A number of changes were made to the default AJP Connector configuration in 9.0.31 to harden the default configuration. It is likely that users upgrading to
---	----------------------------	---

5. Mitigation Plan

Based on the interpretation of the results from the manual review and static testing report I have identified steps to mitigate the identified security vulnerabilities by creating an action list that documents how to fix each vulnerability in my vulnerability assessment report.

Input validation:

- Validate the input parameters to the content and content2 fields in the CRUD class.
- Validate the input parameter to the CRUDController.read() method.
- Validate the input parameter a in the deposit() method of the Customer class to guarantee it's a non-negative value.
- Validate the input parameters key and value in the read_document() method of the DocData class.
- Validate the input parameters id and content in the Greeting constructor.
- Validate the input parameter name in the GreetingController.greeting() method.

Cryptography:

- Encrypt the content and content2 fields in the CRUD class.
- Encrypt the customer's account number and account balance in the Customer class.
- Store the connection credentials to the database in a secure location, such as an environment variable or a secrets management system.
- Encrypt the content field in the Greeting class.

Client/server:

- Implement authentication and authorization checks in the getContent() and getContent2() methods in the CRUD class.
- Implement authentication and authorization checks in the CRUDController.read() method.
- Only reveal the customer's account number in the showInfo() method of the Customer class when the user's authentication and authorization are confirmed.
- Implement authentication and authorization checks in the read_document() method of the DocData class.
- Implement authentication and authorization checks in the GreetingController.greeting() method.

In addition to the above recommendations, I would also recommend using a security framework such as Spring Security to implement authentication, authorization, and other ready to use security features.